Assignment 2 Reports - FIT2099

Written by:

Nguyen Khang Huynh - 33326460

Ngo Tran Ngoc Duy - 33192456

Fabian Wise - 32597223

Table of Contents

Requirement 1

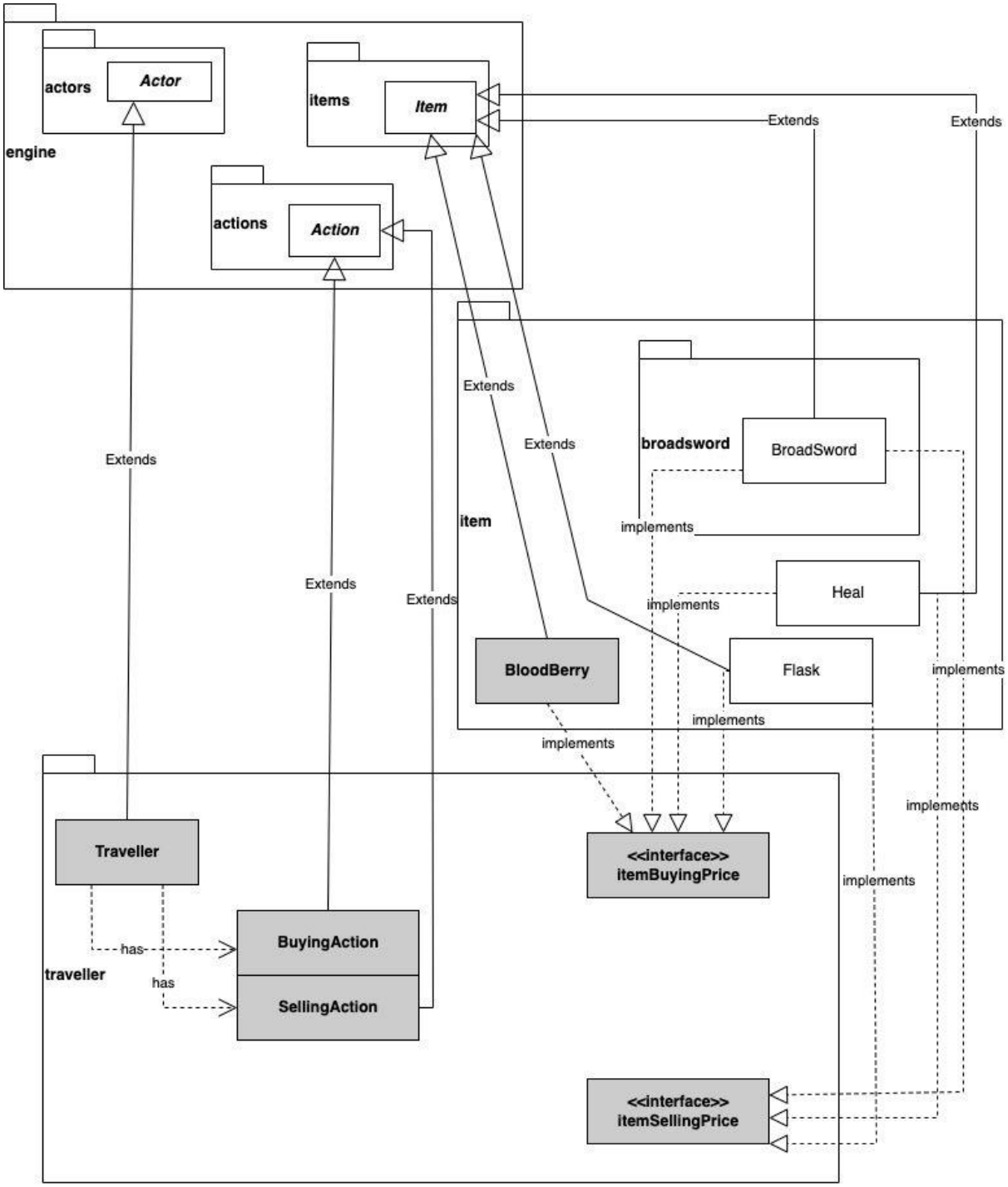

Photo 1: Requirement 1 UML

For REQ1 I used templates of the WanderingUndeadFactory and WanderingUndead to create the RedWolf and ForestKeeper. This was to both make the design simpler to implement and ensure a compatible design with interfaces and extensions, as well as interfacing to the existing game engine. The FollowBehaviour was done by modifying the existing FollowBehaviour in the Mars Demo. The dropping of HealingVial was also done by using the existing templates of WanderingUndead.

Requirement 1 was simple, it was really just using existing objects and modifying for the different requirements in the assignment.

Requirement 2

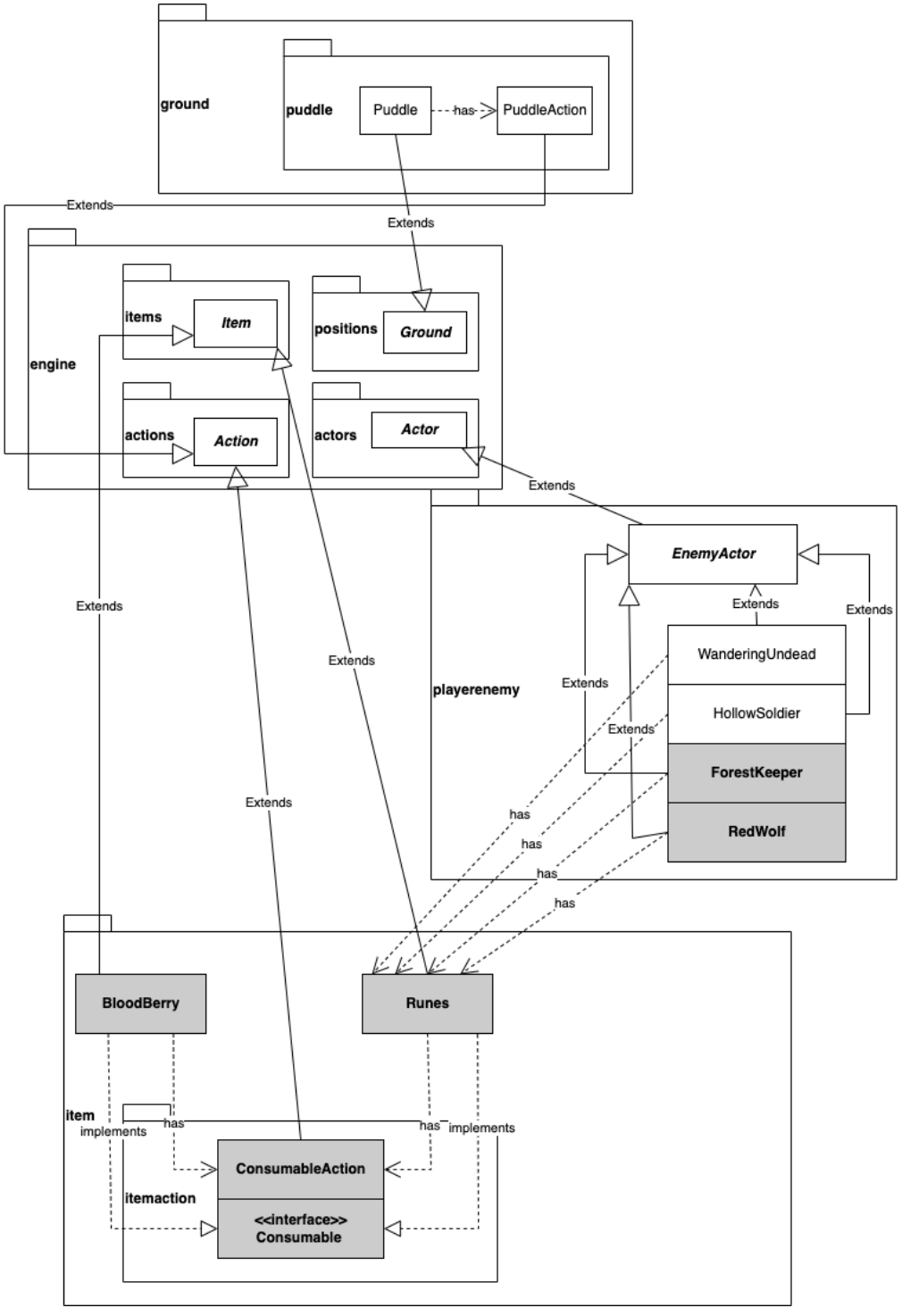

Photo 2: Requirement 2 UML

In requirement 2, we did implement the Blood Berry which extends the abstract class Item and implement some interfaces like Consumable and Item buying price. We implement the interface Consumable to update the status of the player after they used one of the items. The Item buying price I implemented for the traveller which indicates the price they bought that item and the item selling price indicates the price that traveller sells to actors. The Blood berry class as it extends the Item Abstract class so it has to take those three attributes which are name, display char and portable from the Item.

Besides, we did implement the class Runes which also extends Item and implements the interface Consumable so Runes also has 3 attributes from the Item which are name, display char, portable. We did override the method consumed from that interface, like I said above, that interface updates the actor status after using the item, here it updates the actor's balance.

Lastly, about the Puddle, we just extend that from the Abstract class Ground so puddle will take the attribute display char from Ground. And about the allowable actions, if the player stands in that puddle, it will go to the puddle action class where updating the player's stamina by 1% of their maximum stamina.

The special thing here is that we implemented the Item buying and selling price so that in those items like Flask, Heal we just need to override the function's original buying and selling price then return its buying price and the price the traveller sells to actors. And in the rest requirements or in the next assignment, if there are more items, we can still add their price into those two interfaces.

Requirement 3

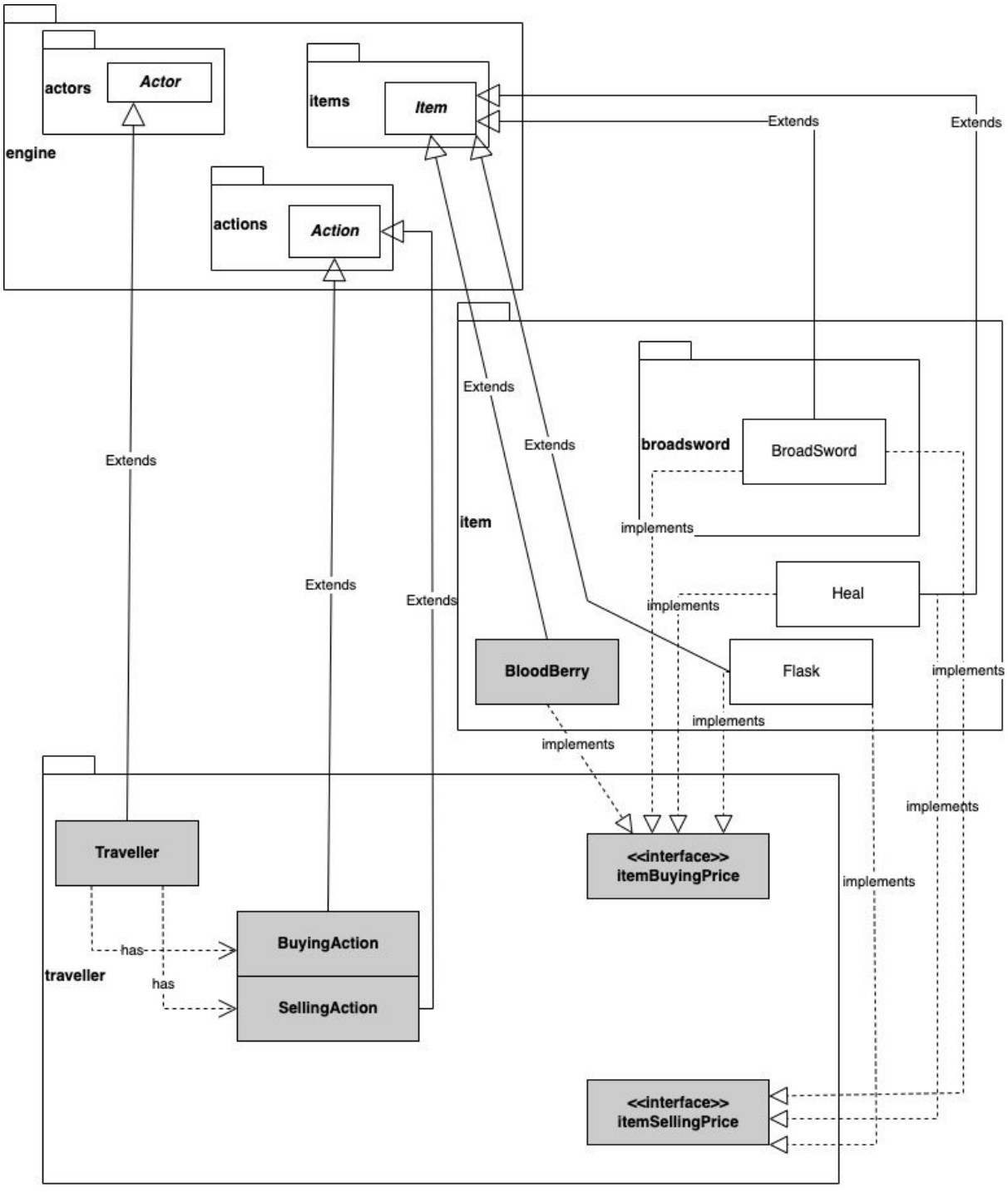

Photo 3: Requirement 3 UML

For task 3, we did implement the traveller that extends the abstract class Actor of course as it is an actor, and it takes name, display char and hit points from the Actor and the traveller can sell the Healing vial, refreshing flask and the broadsword so we put those in the allowable actions which we override it from the Actor. Because of that, the traveller is dependent on the Buying Action and Selling Action. And we also add the Play Turn function which we override it from the Actor that indicate that the traveller does nothing in each turn

For the Blood berry class, we did extend it from the abstract class Item then we implement the interface Item buying price to return the original the buying chance and original buying price

I add all the Buying action and Selling action Traveller into the traveller package so that the user can understand the action of traveller easier and maintain the code for traveller easier.

I also upgraded the Item Buying Price and Item Selling Price, with 2 interfaces, we can check if the Buying chance is true then return the Buying Price (function at the top) to apply to the actor. This can also be taken advantage of requirement 4 when implementing the Great Knife and Giant Hammer.

For the update of 22/09, I had to create another constructor as it says that items will have fixed prices for different traders. I created a hashmap to store different items and different prices so that different traders selling the same item can have different prices.

Requirement 4

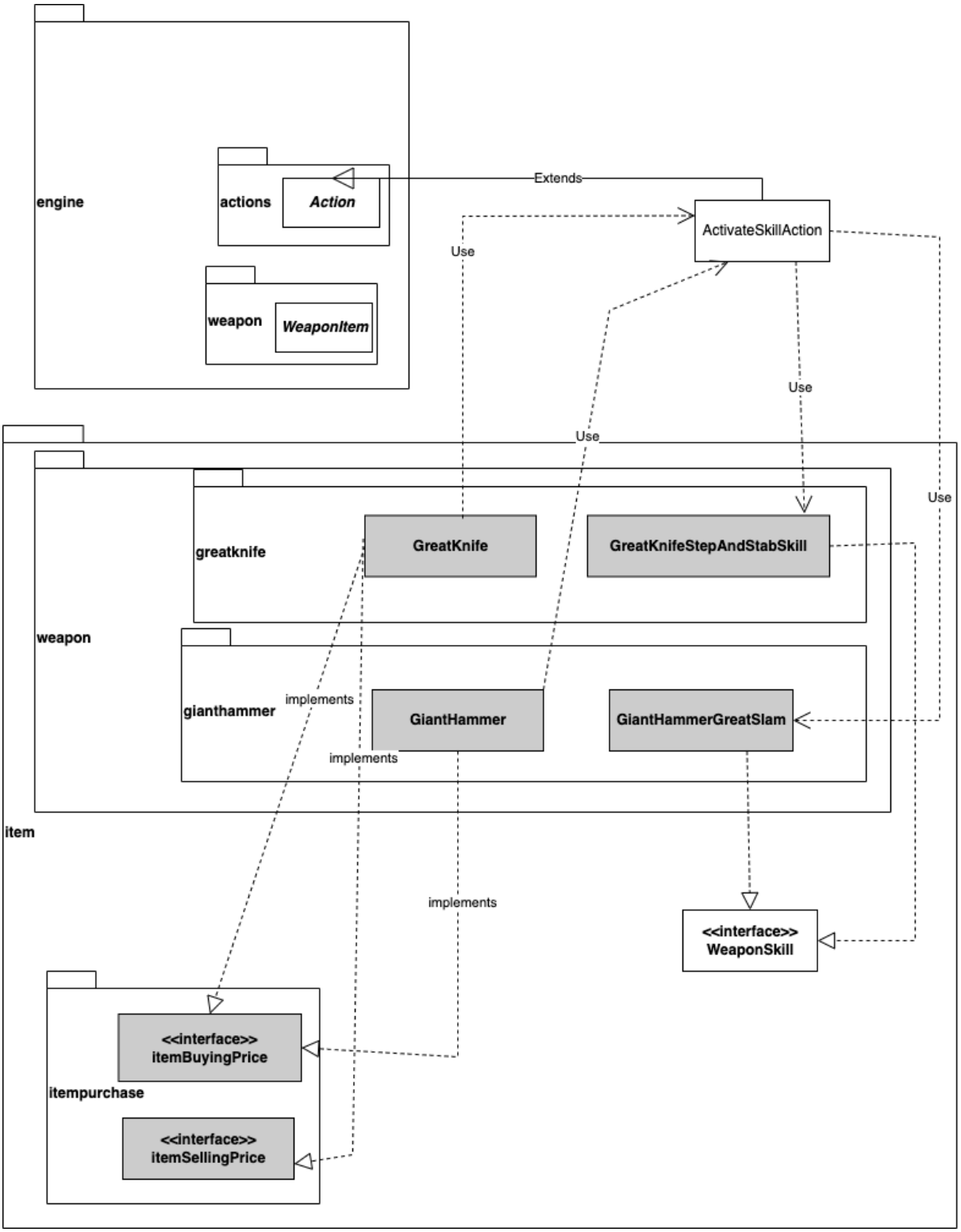

Photo 4: Requirement 4 UML

The price of the Great Knife is set to 300 runes and when the player purchases a great knife, there is a 5% chance that the traveller will ask the player to pay 3x the original price of the weapon, we did these by taking advantage of the Item Buying Price and Item Selling Price which will increase the code readability and reusability. This special skill consumes 25% of the player's maximum stamina which we modify the Activate Skill Action class so that when the special skill is activated it could do that. We do it the same with the Giant Hammer. We also fix the Activate Skill Action and make use of the interface Item Buying Price which can return the chance when they buy, if it is true then return the updated price, if not then return the original price. One thing to note is that when implementing the Giant hammer, its special skill did affect the surroundings of the actor performing it as well as the actor performing it.

Requirement 5

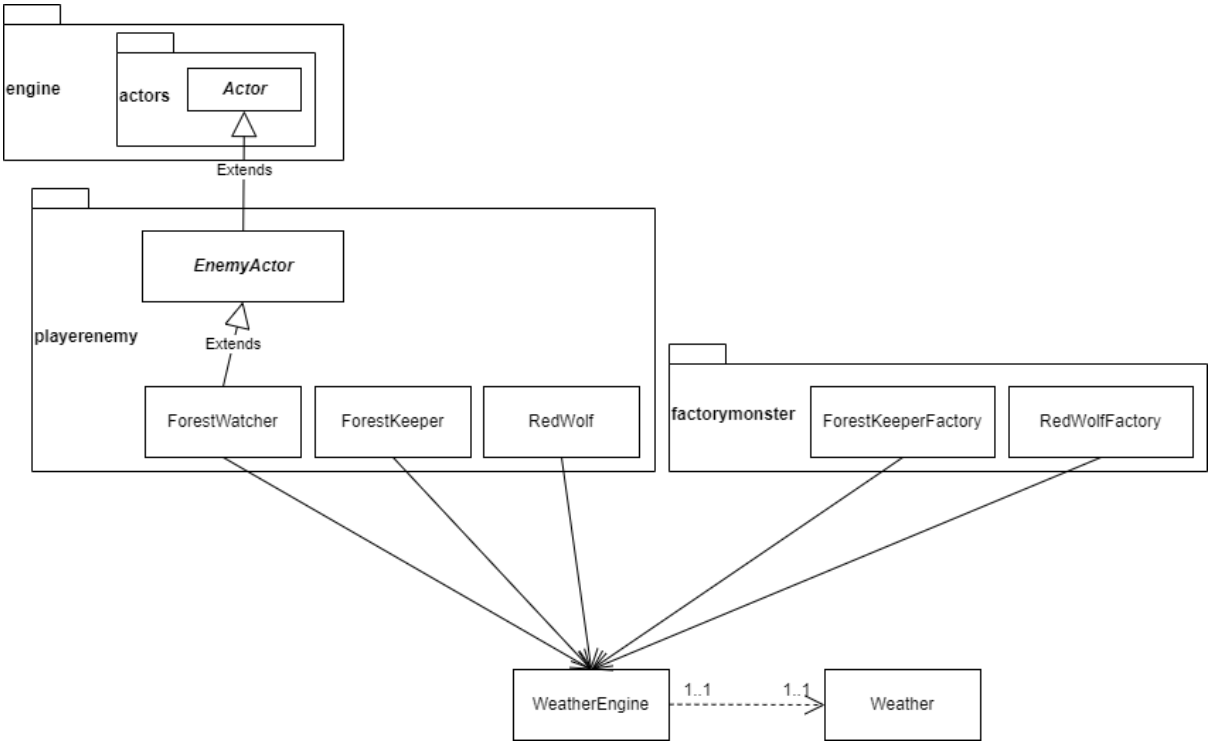

Photo 5: Requirement 5 UML

Weather was added by making an enum type to denote whether it's SUNNY or RAINY. A class called WeatherEngine was created to add and check enum types.

WeatherEngine was created in the Application layer of the program. For objects that require the weather to determine its behaviour, the WeatherEngine object is passed through all of them, simulating a global variable, but it's passed through as parameters. This means that if the weather changes in the base application layer, it also changes the weather for all the objects that use it.